C语言——面向过程编程

单元三、项目重构

第二次课 函数指针与*typedef*

一、函数指针

① 函数名的本质

函数名代表了函数代码在内存中的地址，像数组名一样；也是一个地址常量。

(全局范围)

```
#include <stdio.h>

int add_int(int a, int b);


int main    ()
{
    printf("%d\n", add_int);
    printf("%p\n", add_int);
    return 0;
}

int add_int(int a, int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

打印函数名：会看到其内存地址

C:\WINDOWS\system32\cmd.exe

```
5771554
00581122
请按任意键继续. . .
```

② 对函数名取地址

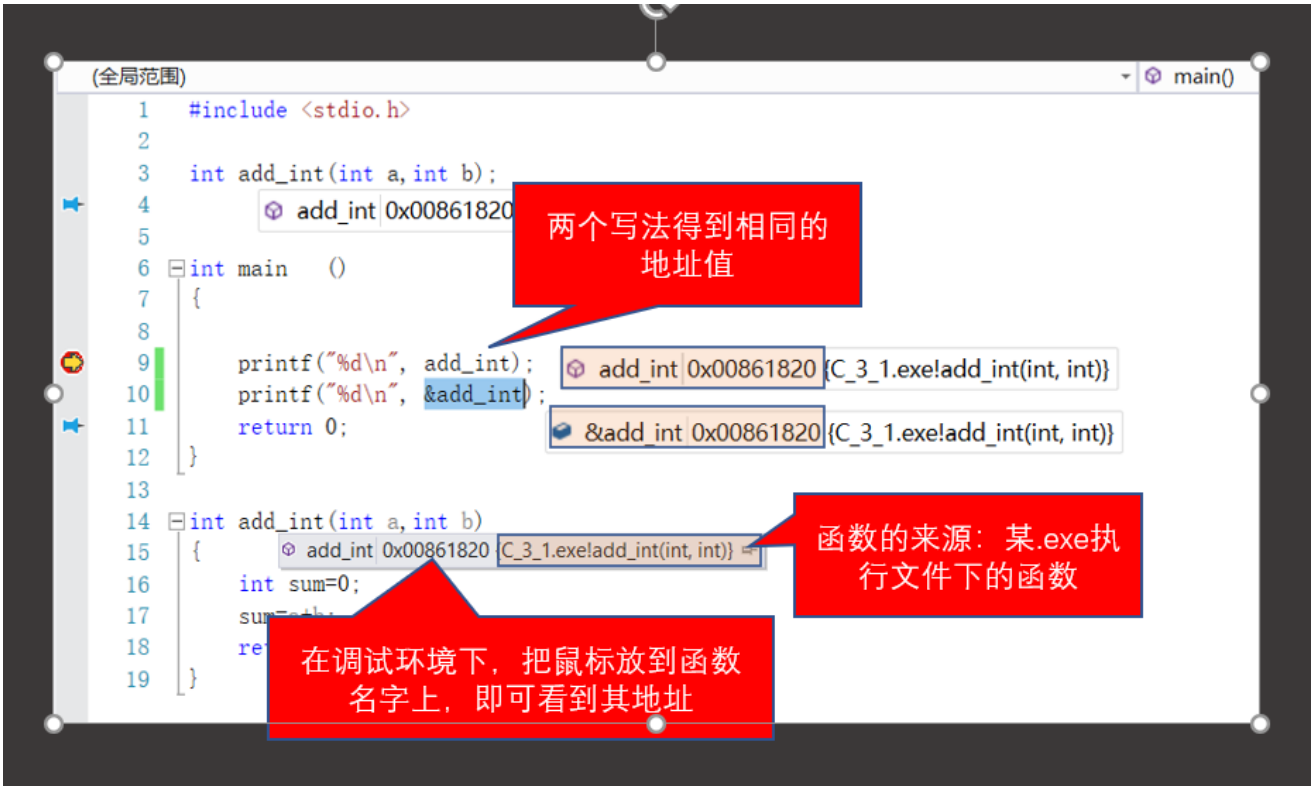
```c
#include <stdio.h>

int add_int(int a,int b);


int main    ()
{

    printf("%d\n", add_int);
    printf("%d\n", &add_int);
    return 0;
}

int add_int(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

对函数名取地址得到相同的地址

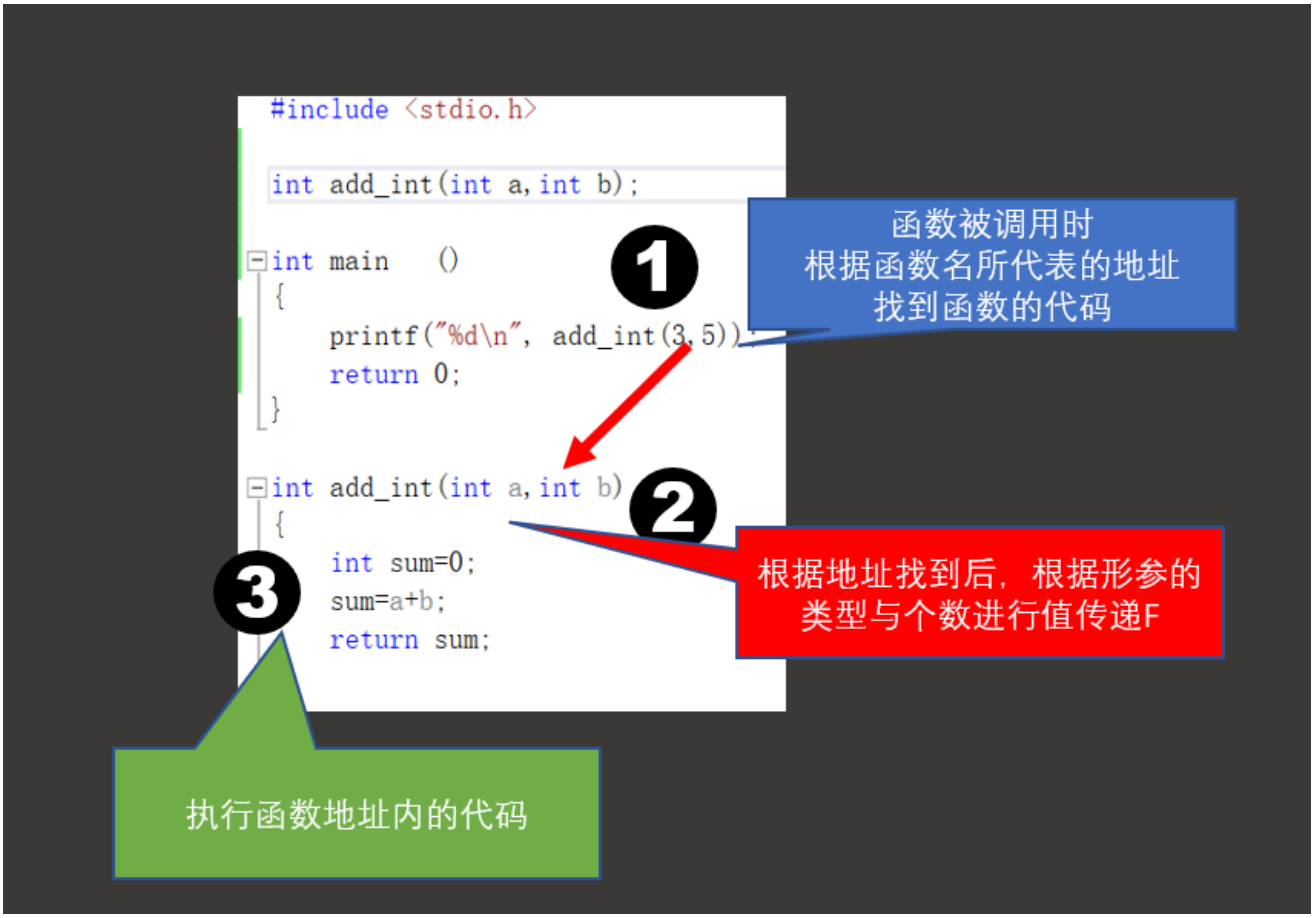
C:\WINDOWS\system32\cmd.exe

```
12521762
12521762
请按任意键继续. . .
```

③通过调试器查看



④函数被调用时，能够执行的本质：地址+实际参数



⑤ 函数指针的声明

既然函数名是地址，就可以用指针变量来保存。根据之前指针的使用经验：指针变量必须强调是什么类型的指针。因此，函数的指针定义时必须把函数的特征表述清楚。就如同一维数组指针与二维数组指针的定义一样：回忆如下代码

```
//一维数组与指针
int a[5];// 一维数组的特征 int  [5]
int (*p)[5];//一维数组指针保留特征 int [5] 然后在 名字位置上写成：(* 指针变量名)
//二维数组与指针
int arr[5][3];// 二维数组的特征 int  [5][3]
int (*p)[5][3];//二维数组指针保留特征 int [5][3] 然后在 名字位置上写成：(* 指针变量名)
```

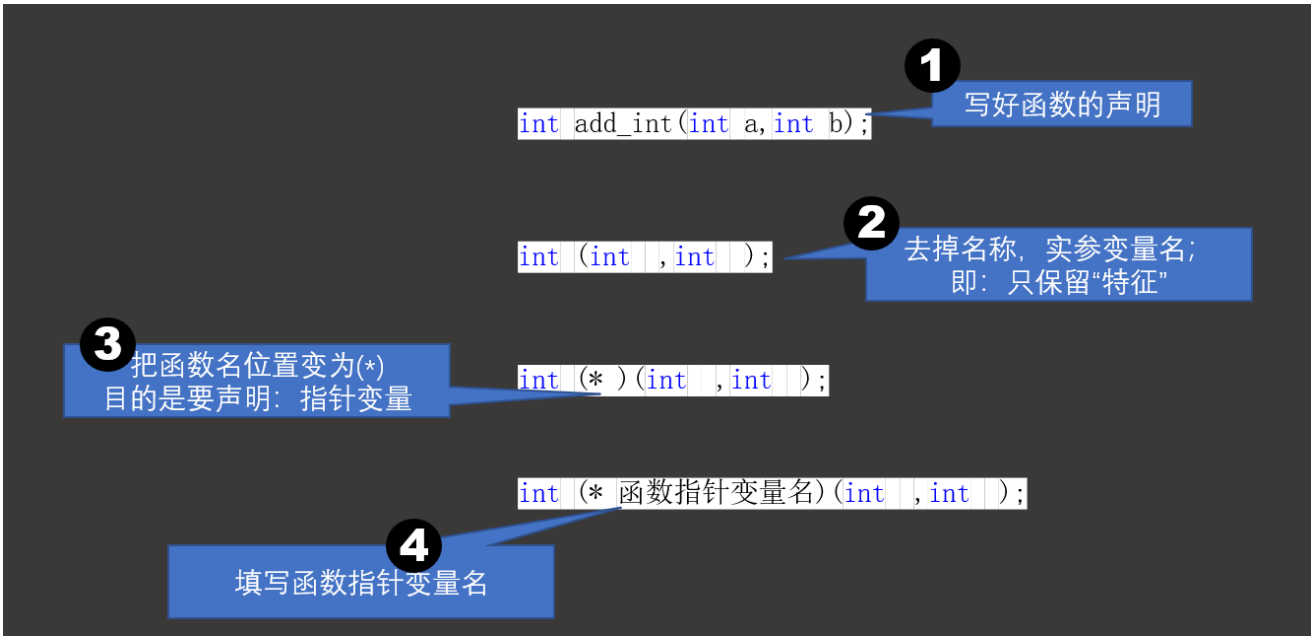
基于以上模式，函数指针的声明方式：函数特征+(*指针变量名)

其中函数特征包括：返回值类型， 参数个数及每个参数的类型不理会实参变量名具体是什么。如：

```
#include <stdio.h>

int add_int(int a,int b);

int main   ()
{
    int  (* pfun)(int  ,int  );//声明一个能指向add_int的函数指针
    //为函数指针赋值，使其能指向函数代码，并能执行所指向的函数代码。
    pfun=add_int;//方式1：把函数名赋给函数指针变量
    pfun=&add_int;//方式2：把函数名先取地址，再把计算的地址赋给函数指针变量
    return 0;
}

int add_int(int a,int b)
{
```

```
    int sum=0;
    sum=a+b;
    return sum;
}
```

【注意：】定义函数指针变量的过程



⑥ 利用函数指针执行函数代码。

方式1：(*函数指针) (实际参数) ← 推荐

方式2： 函数指针 (实际参数)

```
#include <stdio.h>

int add_int(int a,int b);

int main   ()
{
    int  (* pfun)(int  ,int  );//声明一个能指向add_int的函数指针
    //为函数指针赋值，使其能指向函数代码，并能执行所指向的函数代码。
    pfun=add_int;//方式1：把函数名赋给函数指针变量
    pfun=&add_int;//方式2：把函数名先取地址，再把计算的地址赋给函数指针变量
    //利用函数指针变量 pfun执行其所指向的代码。
    printf("%d\n", (*pfun)(10,20));//显式的使用，让人一目了然它是一个函数指针
    printf("%d\n",pfun(11,22));//隐式的使用，让人看不出它是否是函数指针，字面上与下行代码无异
    printf("%d\n",add_int(111,222));//通过函数名调用函数
    return 0;
}

int add_int(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

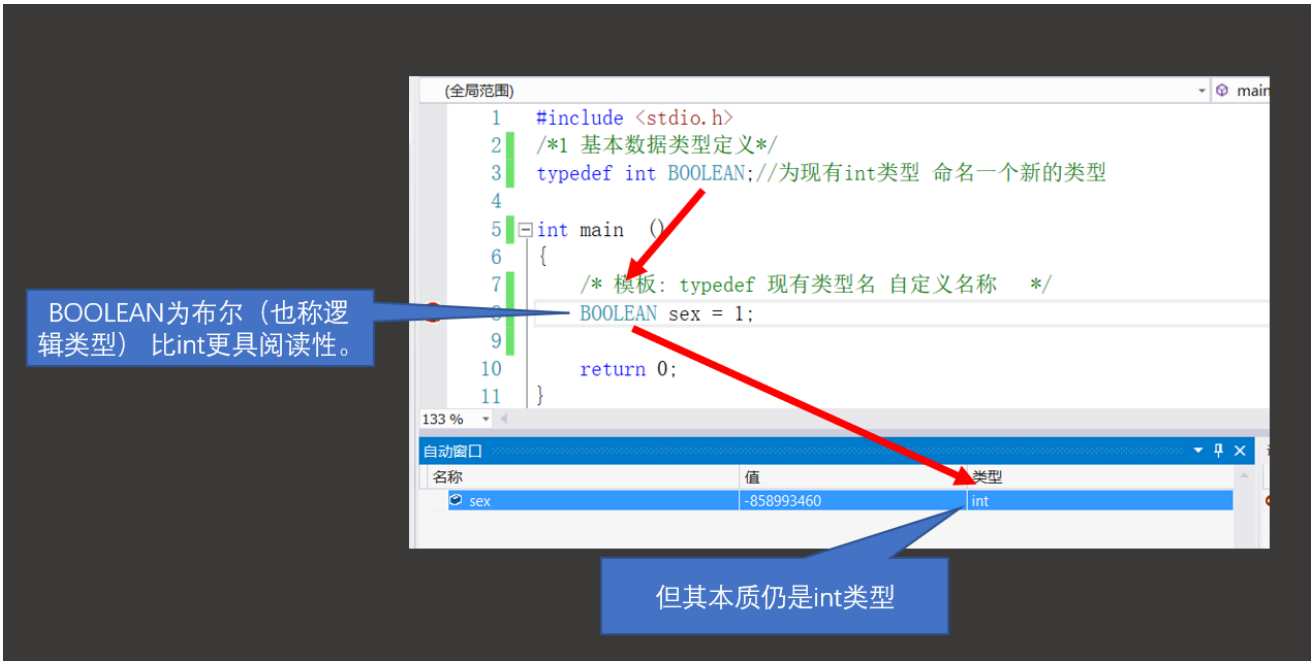
二、typedef 的用法

typedef直译为：类型的预定义。简单来说就是为现有数据类型命一个新的名称，方便阅读与编程语义的流畅性。

其格式为： typedef 已有类型 自定义类型名；

① 基于基本数据类型定义一个新的类型名

```
#include <stdio.h>
/*1 基本数据类型定义*/
typedef int BOOLEAN;//为现有int类型 命名一个新的类型 通常自定义类型名称为大写

int main ()
{
    /* 模板: typedef 现有类型名 自定义名称   */
    BOOLEAN sex = 1;

    return 0;
}
```



② 简化不同维度数组的指针变量定义

```
#include <stdio.h>
/*2  指针类型定义*/
typedef int *  POINT; //列指针
typedef int (*LINE_POINT)[4]  ;//行指针
typedef int (*FLAT_POINT)[3][4]  ;//平面指针

int main ()
{
    int a=99, arr[4]={1,2,3,4},arr2[3][4]={5,6,7,8,9,10,11,12};

     POINT p=&a;//对比 原来的写法
     int* p2 = &a;
     LINE_POINT lp=&arr;
     int (*lp2)[4]=&arr;
     FLAT_POINT fp=&arr2;
```

```
     int (*fp2)[3][4]=&arr2;
    //利用指针取不同列的int型数据
    printf("%d %d\n", *p,*p2);
    printf("%d %d %d\n",  **lp , *(*lp+1) ,(*lp)[2]);
    printf("%d %d %d\n",  ***fp , *( *(*fp+0)+1) ,(*fp)[0][2]);
    return 0;
}
```

③ 简化函数指针的变量定义

```
#include <stdio.h>

int add_int(int a,int b);
//3 定义函数指针 类型
typedef int (*FUN_POINT)(int ,int  );
int main    ()
{
    //对比如下函数指针变量的定义
    int  (* pfun)(int  ,int  ) = &add_int;
    FUN_POINT fp = &add_int; //更简洁直接

    //用函数指针执行函数
    printf("%d\n", (* pfun)(1,2));
    printf("%d\n", pfun(1,2));
    printf("%d\n", (* fp)(1,2));
    printf("%d\n", fp(1,2));
    return 0;
}

int add_int(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

④ typedef嵌套定义新类型

所谓嵌套定义：即在已经定义的类型基础上进行二次定义新类型。

```
#include <stdio.h>
/*4  嵌套定义 */
typedef int INTEGER;//新类型
typedef INTEGER* POINT_INT;//为新类型指针 创建另一个新类型
typedef POINT_INT* POINT_POINT_INT;

int main()
{
    INTEGER a=10;
    POINT_INT p=&a;
    POINT_POINT_INT pp=&p;

    printf("%d\n",a);
    printf("%d\n",*p);
    printf("%d\n",**pp);
    return 0;
}
```

三、函数指针的应用场景（模拟）

① 模拟计算器的屏幕显示：

```
#include <stdio.h>
typedef int (*COMPUTE)(int  ,int  ) ;
int jia(int a,int b);
int jian(int a,int b);
int cheng(int a,int b);

/*模拟计算器打印的屏幕*/
void print(int a,int b,COMPUTE pf );
int main()
{
    /*调用自定义函数print
        第三个实参是函数名，print函数的形参COMPUTE pf 就执行实参函数
        通过函数指针就可执行函数代码。
        函数指针增强了代码的灵活性。
    */
    print(99,66,jia);
    print(99,66,jian);
    print(99,66,cheng);
    return 0;
}

/*加法*/
int jia(int a,int b)
{
    return a+b;
}
/*减法*/
int jian(int a,int b)
{
    return a-b;
}

/*乘法*/
int cheng(int a,int b)
{
    return a*b;
}
/*以打印模拟屏幕结果的显示*/
void print(int a,int b,COMPUTE pf )
{
    char s;
    if(pf==jia)
    {
        s='+';
    }
    else if(pf==jian)
    {
        s='-';
    }
    else if(pf==cheng)
    {
        s='*';
    }

    printf("%d %c %d =%d\n",a,s,b,pf(a,b));
```

```
}
```

② 模拟网络连接、服务器响应后的回调机制

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
typedef int DATA;
typedef char* STRING;
typedef void (*CALLBACK) (DATA );
void httpRequest(STRING url ,CALLBACK cb);
void loginAction (DATA data );
void openPage (DATA data );
int main()
{
    //模拟：打开不同的网址，做不同的回调处理
    /*
        所谓回调：当客户端打开一个网址，从路由连接到服务器处理，再把处理结果带回是需要等待的。
        因此 函数loginAction 、openPage需要服务器处理完再执行。这种等待网络连接处理后再执行的函数
        称为回调函数。——本例为模拟过程，绝非真实网络连接。
    */
    httpRequest("http://www.baidu.com",loginAction );
    httpRequest("http://www.toutiao.com",openPage );
    return 0;
}
/*模拟http的网络请求*/
void httpRequest(STRING url ,CALLBACK callbak)
{
    DATA response;
    //模拟网络连接
    printf("正在连接：%s\n",url);
    printf("等待服务器处理...\n" );
    //用随机数模拟服务器处理结果
    srand(time(NULL));
    response = rand()%10;
    callbak(response);
}
/*模拟登录处理*/
void loginAction (DATA data )
{
    if(data )
    {
        printf("欢迎进入...\n");
    }
    else
    {
        printf("登录失败 权限不足\n");
    }
}
/*模拟打开页面处理*/
void openPage (DATA data )
{
     int i;
     printf("你正在浏览新闻：\n");
     for(i=1;i<=data;i++)
     {
        printf("第%d行新闻\n",i);
     }
}
```